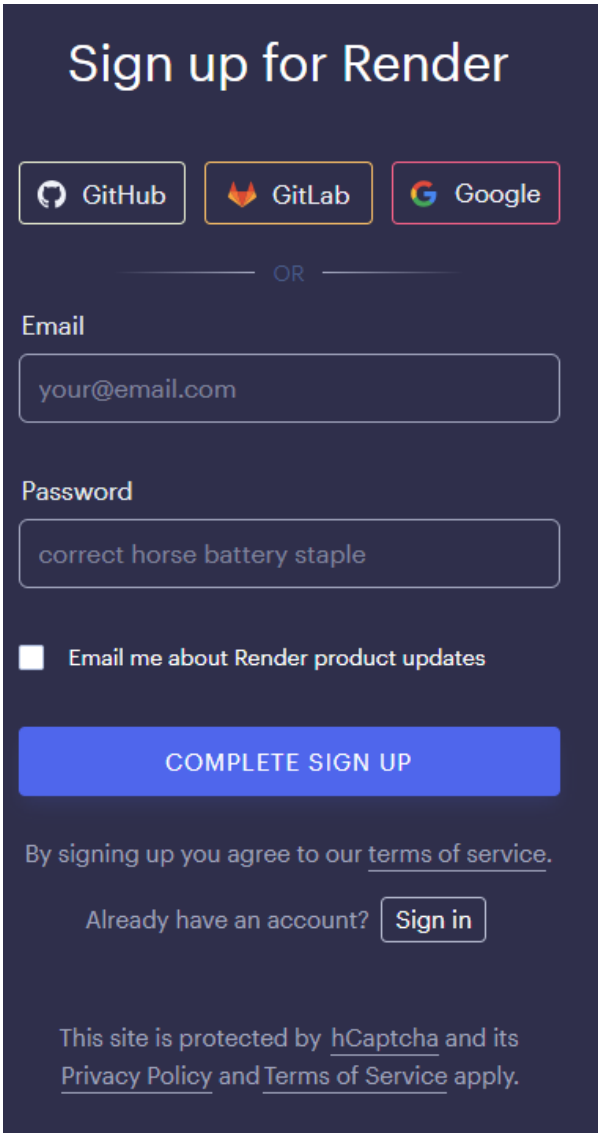
Launch a Node.js Website with Render and GitHub

Introduction:
The goal of this document is to serve as a walkthrough of the creation of a Render Account and launching a simple Node.js website using code from a GitHub Repo.

Render is a web hosting platform supporting multiple programming languages and software. It will let us run Node.js code, connect to MySQL and MongoDB databases and quickly and automatically deploy code using Git version control. And the best part? Most of these services have a free-tier to you let you try it out!

Step 1: Create a Render Account.
Go to https://dashboard.render.com/register

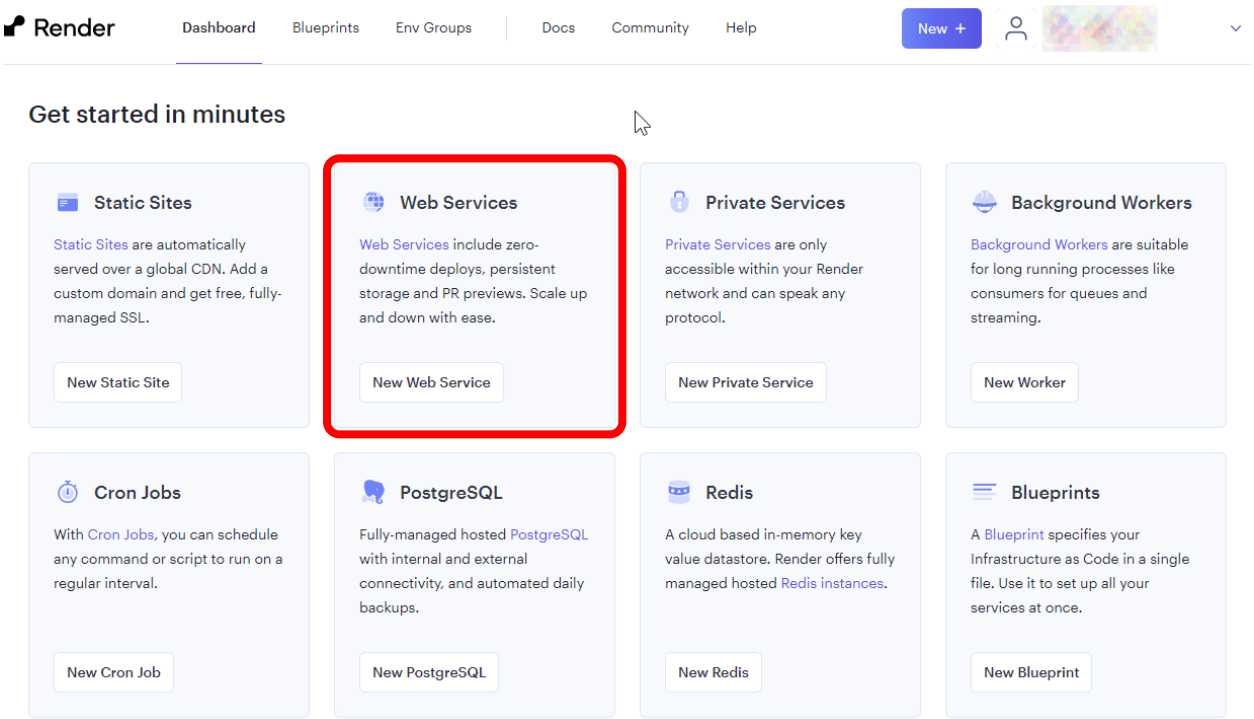
Fill in the necessary details to create an account and verify your account by clicking the link sent to your email.



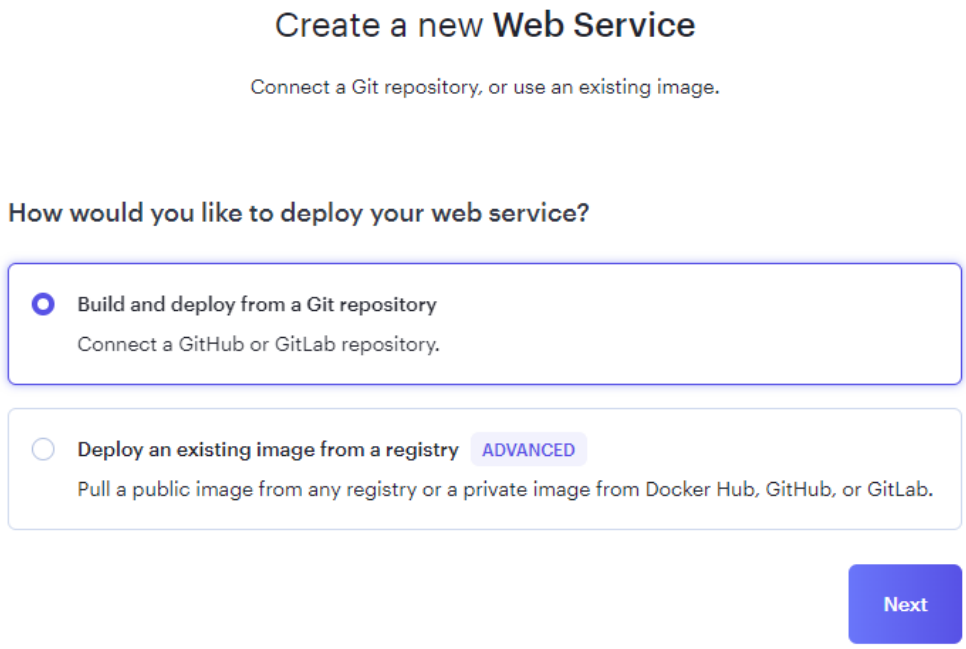
Step 2: Create a new Render app.

Once you have created your Render account, create a new Render app.

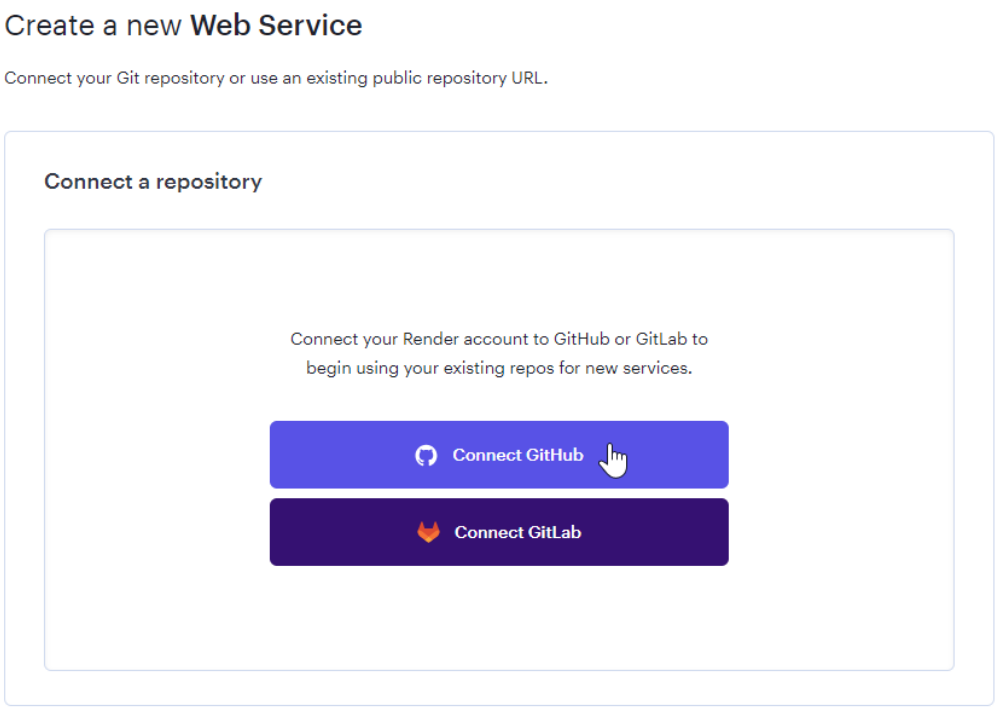
Click on "Web Services" to start.



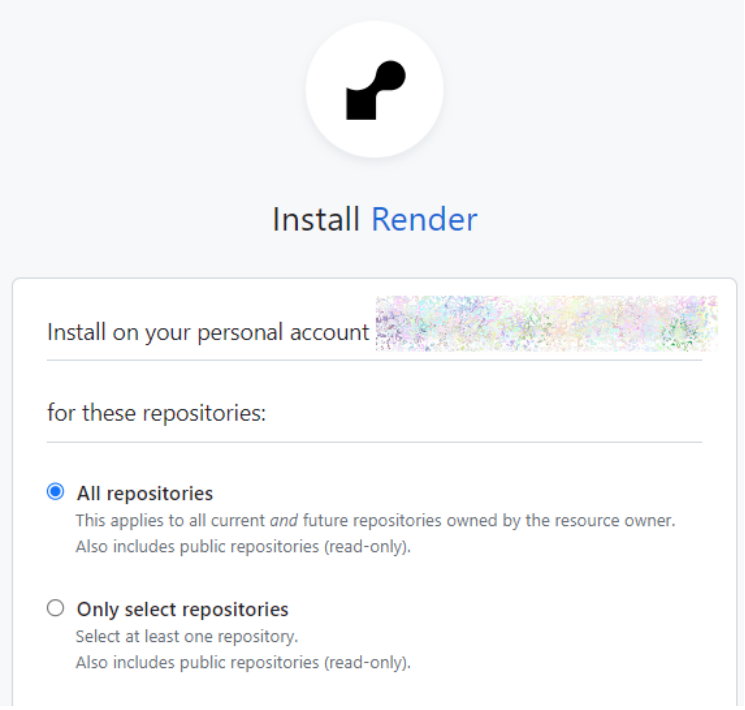
Click on "Build and Deploy from Git repo":



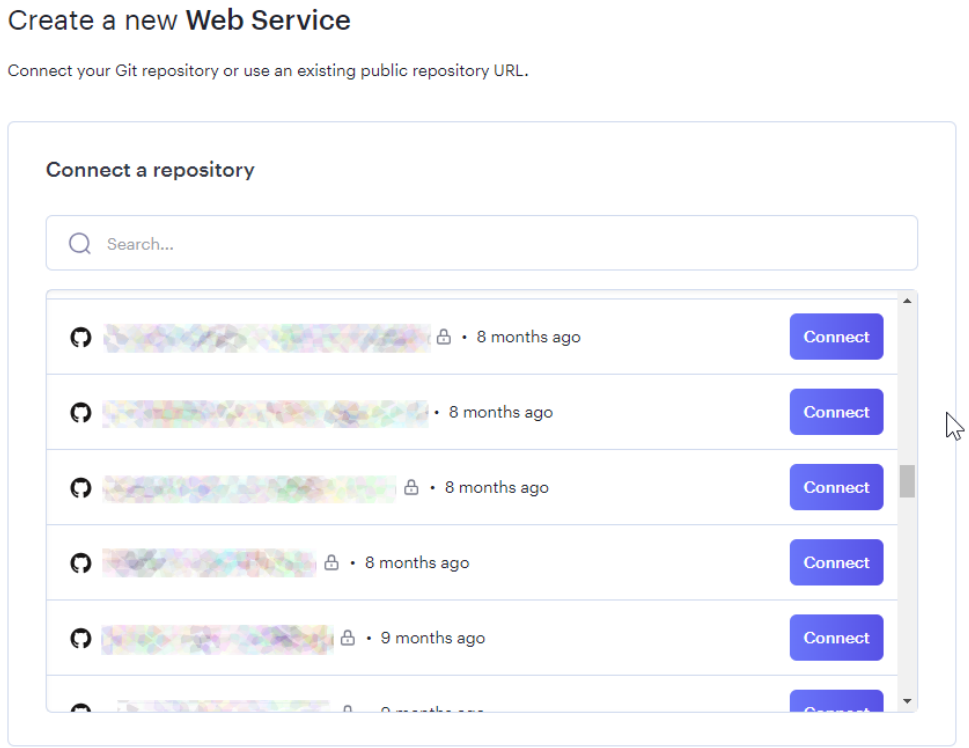
Click "Connect to GitHub":



Authorize Render to have access GitHub (this step should only be required once):



Once you have given Render permissions to access your GitHub, you should see a list of your existing GitHub Repos:



Click "Connect" next to the repo you wish to host. This tutorial assumes the repo contains a website written using Node.js.

You are deploying a web service for

Name
A unique name for your web service.

forgetable-elephant-crush

Region
The region where your web service runs.

Oregon (US West)

Branch
The repository branch used for your web service.

main

Root Directory Optional
Defaults to repository root. When you specify a root directory that is different from your repository root, Render runs all your commands in the specified directory and ignores changes outside the directory.

e.g. src

Runtime
The runtime for your web service.

Node

Build Command
This command runs in the root directory of your repository when a new version of your code is pushed, or when you deploy manually. It is typically a script that installs libraries, runs migrations, or compiles resources needed by your app.

```
$ npm i
```

Start Command
This command runs in the root directory of your app and is responsible for starting its processes. It is typically used to start a webserver for your app. It can access environment variables defined by you in Render.

```
$ node index.js
```

Give your new app a new name. This name must be unique across all Render applications.

You can use the course name, project name and then your student number:

Example: `comp9876-week1-A87654321`

Select your branch name; this is typically "main", however, it could be another branch if you have multiple branches in your repo.

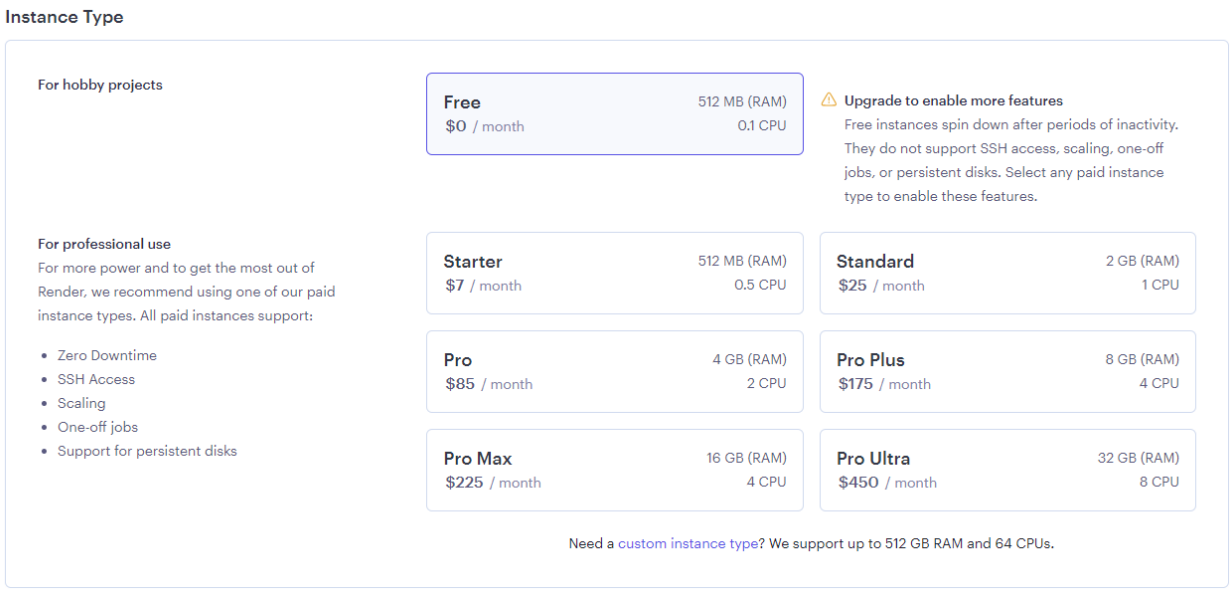
If your project is not in the root of your repo, then you should specify the root directory.

Use the dropdown to select the runtime. In our case, select "Node" so that it can run our Node Server.

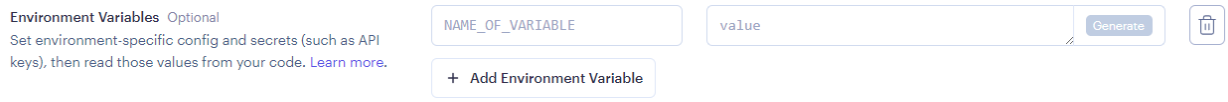
For runtime, specify what command you need to build your project. This is typically `npm i`. But it could be something more complicated if your project needs it.

For start command, you will put in what command you would run on the command line to start your application. For node, this will be `node` and the name of your server file.
Typically something like: `node index.js`.

If you don't want to be charged money for hosting, I recommend selecting the "Free" option for Instance type. The "Free" option will suffice for all projects needed for this course.

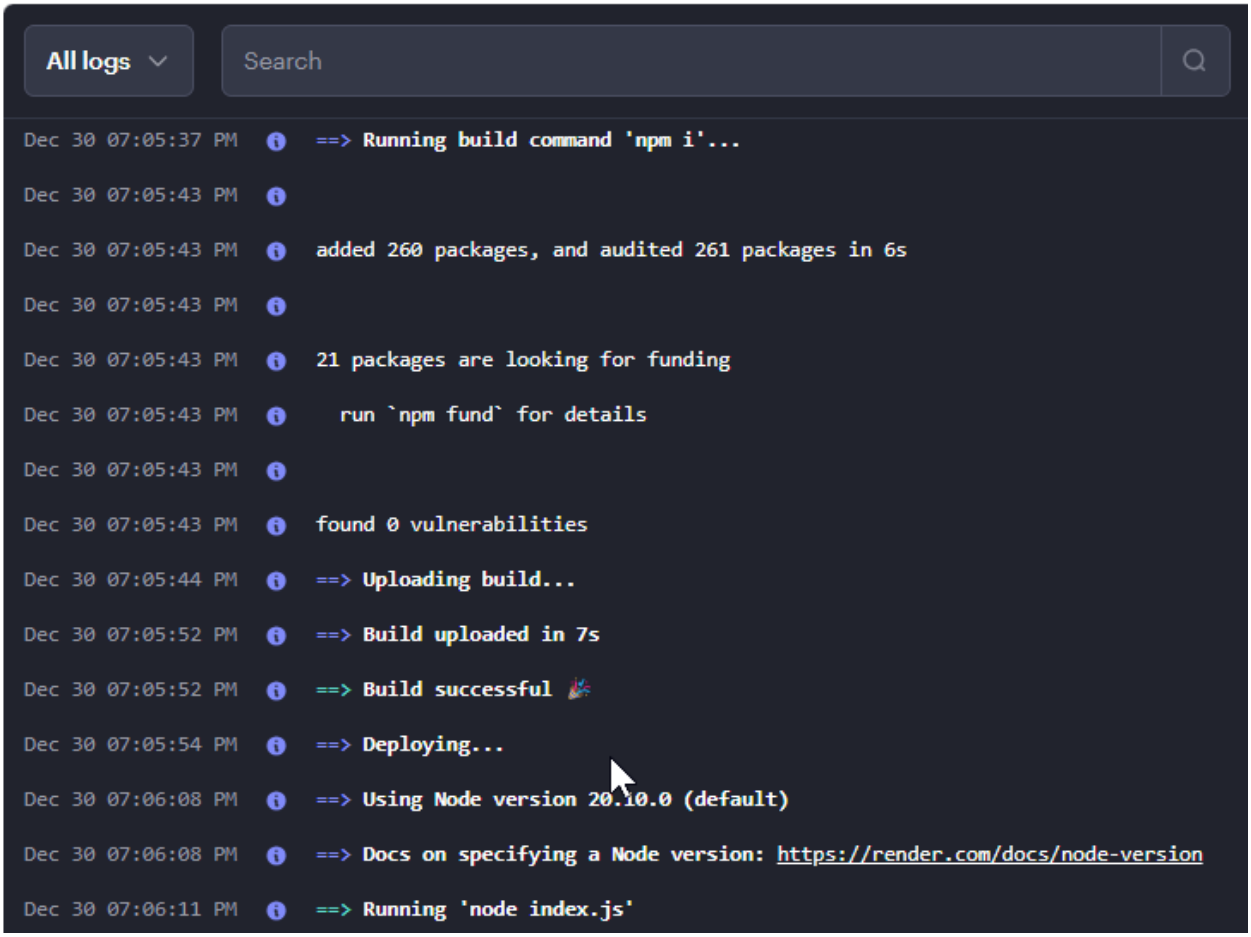


Leave the Environment Variables section blank for now. We will come back to this section after we have completed the initial configuration.



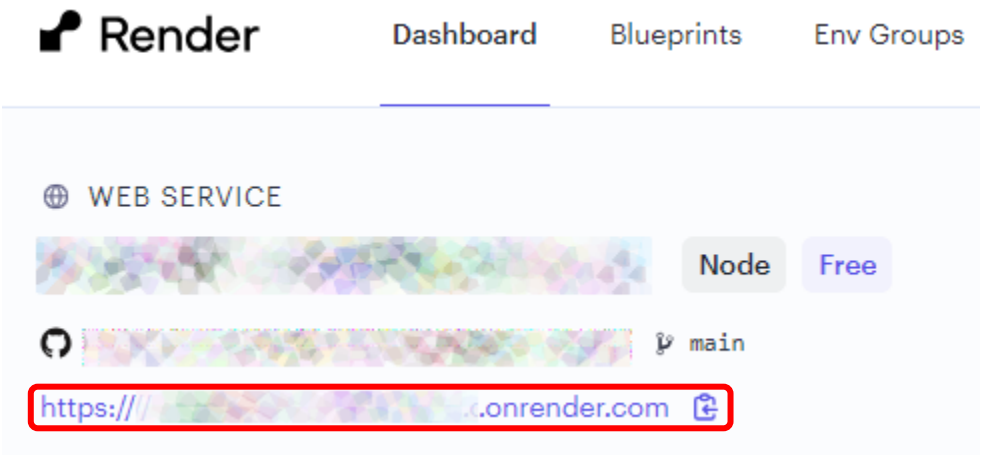
Click on "Create Web Service":

Give Render a chance to build your application. You can see its progress in the log output:

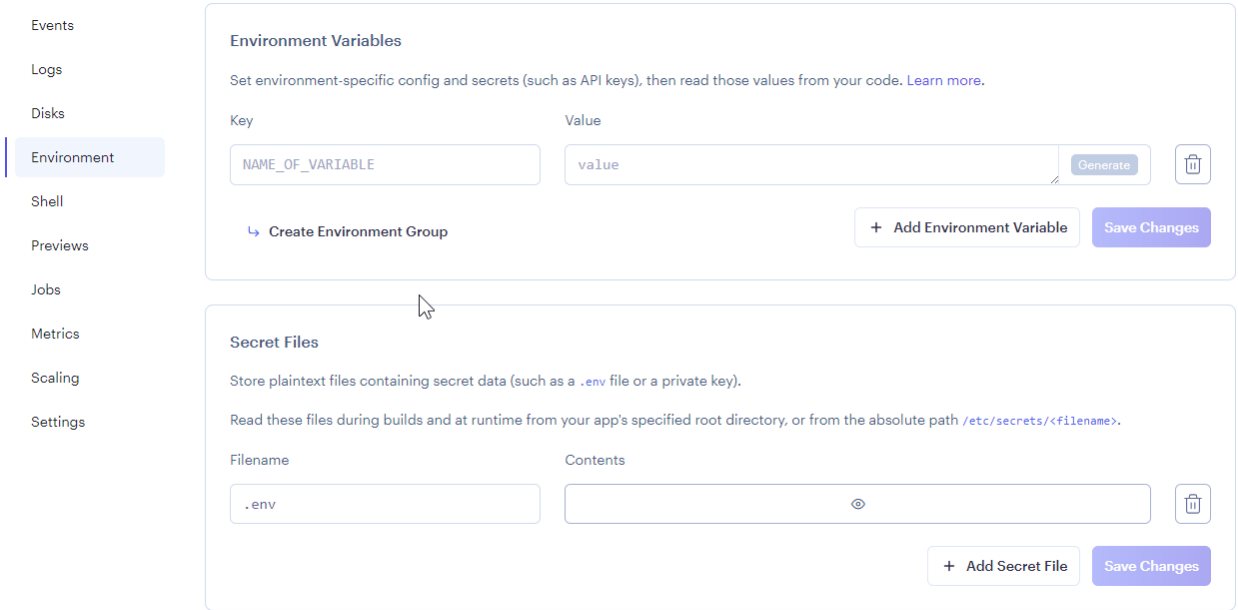


Once the build it complete, you should be able to see your hosted site.

Click the link at the top of your application to see your website.



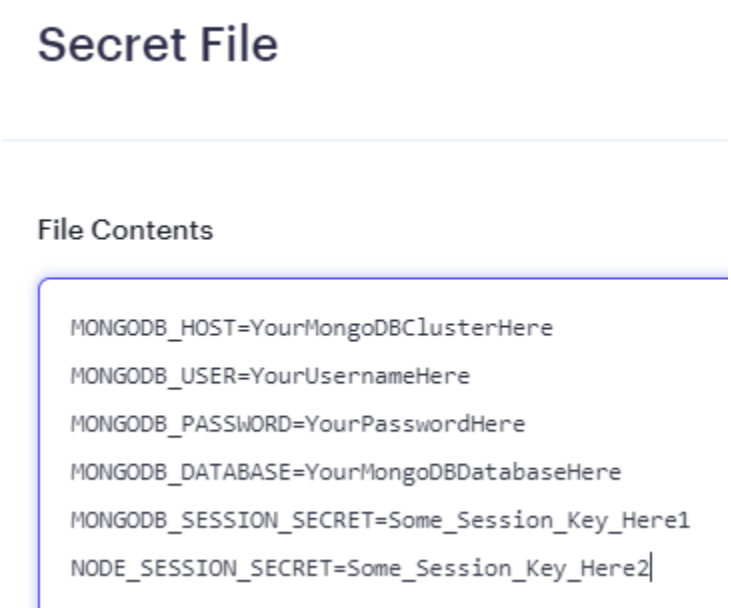
If you have a `.env` file with usernames, passwords and other secrets, you should create a secret file. Remember `.env` files should **NOT** be in your Git repo (they pose a security risk) but are needed to run your application.
If you don't have a `.env` file, you can skip this part.



For example, an .env file like this:

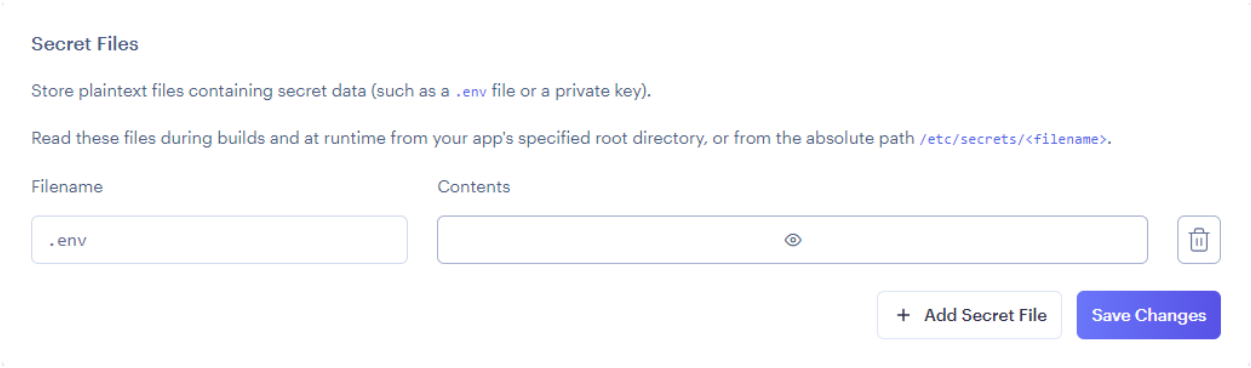
```
MONGODB_HOST=YourMongoDBClusterHere
MONGODB_USER=YourUsernameHere
MONGODB_PASSWORD=YourPasswordHere
MONGODB_DATABASE=YourMongoDBDatabaseHere
MONGODB_SESSION_SECRET=Some_Session_Key_Here1
NODE_SESSION_SECRET=Some_Session_Key_Here2
```

Add a `.env` file to Render as a secret file and paste the contents of the file into the input field, like this:



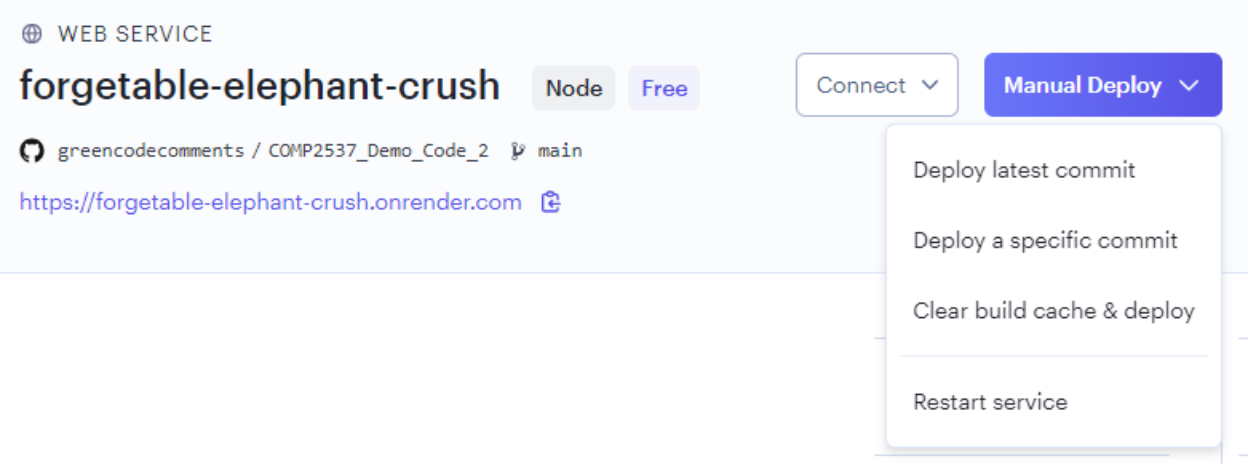
Click "Done" to save your `.env` secret file. Make sure to name the file `.env`.

Click "Save Changes".



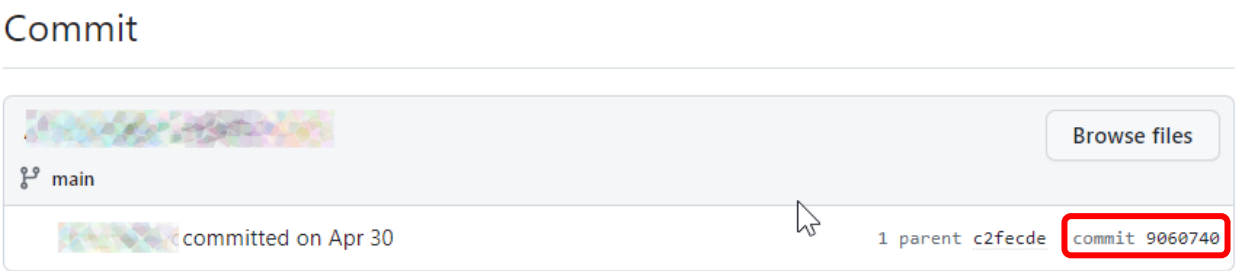
You may need to restart your application for the new environment variables in your secret file to take effect.

Click on the dropdown next to "Manual Deploy" and click "Restart service".

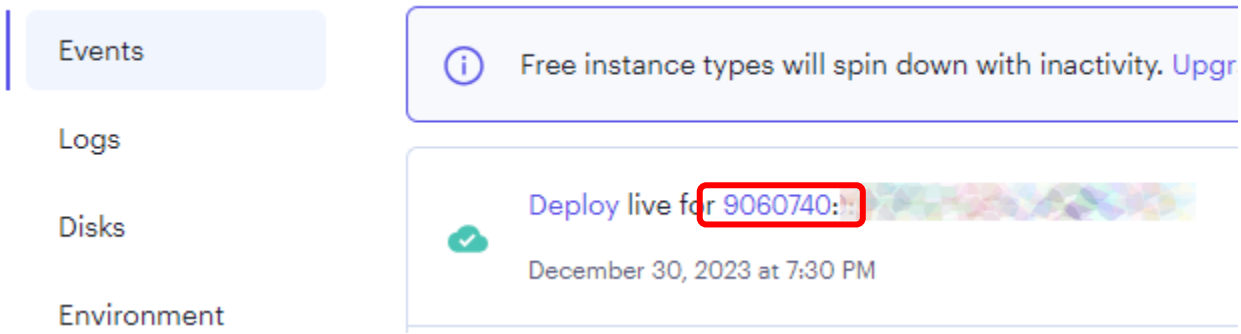


You should take a moment to confirm that the Git commit that Render deployed is the same one as the one you wanted. Each Git commit has a SHA hash that uniquely identifies it.

In the example below, the commit has a SHA hash of **9060740**. This is an abbreviated 7-character hash. A full SHA hash is 40 characters, like this: **9060740**b52bae62e00e086dc4ad14a129186da39. However, usually only the first 7 characters are needed to uniquely identify a commit.



You should confirm that Render has properly deployed the correct commit by looking at Render's Event logs:



(Notice the SHA hash of **9060740** in the Deploy live message above).

One of the best parts about Render is that it watches your Git Repo for changes and will automatically redeploy your application. Your application will always have the latest version of your code on the branch you selected when you linked your repo to Render. Make sure to **commit and PUSH** your files to GitHub, though.